Universidade Federal de Sergipe — **PROCC**
Projeto e Análise de Algoritmos · Período 2026.2
Módulo II — Otimização, Grafos e Processamento de Cadeias

Atividade 2 (AV2) — Relatório Técnico

Recuperação e seleção otimizada de contexto para IA generativa em materiais educacionais abertos

Programação dinâmica e mochila 0/1 para seleção de chunks sob restrição de tokens: um estudo comparativo com estratégia gulosa

João Cosme Sena Sá · Eduardo Henrique do Lago Silva · Helena Carvalho Leal
Hernandison da Silva Bispo · Nadianne Maria dos Santos Galvão · Rafael Takeguma Goto

Tema	8 — Materiais educacionais abertos
Corpus	*Pense Python* (tradução livre da 3ª ed. de *Think Python*)
Repositório	`https://github.com/joaocss/PAA_UFS_2026_2_SenaSa_Joao_Bispo_Hernandison_Galvao_N`
Licença do código	MIT · AV2 em desenvolvimento (tag pendente)
Vídeo da atividade	*Pendente de produção e publicação (AV2)*

Documento revisado em 27 de outubro de 2026 · São Cristóvão, SE
Entrega dos artefatos até 04/11/2026 · Apresentação em 05/11/2026

Estado desta versão do relatório.

Este documento apresenta a evolução do protótipo desenvolvido na AV1, incorporando a seleção de contexto sob restrição de tokens, conforme a Trilha A da Atividade 2.

A abordagem principal será a programação dinâmica para o problema da mochila 0/1, comparada a uma estratégia gulosa por relevância/token.

A especificação formal, a implementação, a análise de corretude, a complexidade e os resultados experimentais serão documentados nas respectivas seções, conforme o desenvolvimento da atividade.

Sumário

1 Identificação da equipe **3**

2 Tema, corpus e fonte **3**

2.1 Continuidade da Atividade 1 4

3 Repositório, licença, versão e data de acesso **4**

4 Especificação do problema **5**

4.1 Definição do problema 5

4.2 Corpus e representação dos chunks 5

4.3 Função de relevância 6

4.4 Definição formal da entrada 7

4.5 Definição formal da saída 7

4.6 Função objetivo 8

4.7 Restrições do problema 8

4.8 Critério de corretude 9

4.9 Linha de base: estratégia gulosa 9

4.10 Casos de borda 10

4.11 Parâmetros experimentais preliminares 11

4.12 Decisões metodológicas 11

5 Representação dos dados e estratégia de chunking **12**

5.1 Normalização 12

5.2 Segmentação (chunking) 12

5.3 Representações computacionais 12

6 Algoritmos e pseudocódigo **13**

6.1 Programação dinâmica: mochila 0/1 . . . 13
6.2 Estratégia gulosa por relevância/custo . . . 13
6.3 Justificativa de corretude . . . 13

7 Análise no modelo RAM **13**
7.1 Parâmetros . . . 13
7.2 Operações elementares . . . 14
7.3 Programação dinâmica: mochila 0/1 . . . 15
7.4 Estratégia gulosa por relevância/custo . . . 17
7.5 Custo do pipeline de recuperação e seleção . . . 19

8 Melhor, pior e caso médio **22**

9 Recorrências **23**
9.1 Programação dinâmica: mochila 0/1 . . . 23
9.2 Relação com o custo computacional . . . 24
9.3 Recorrência temporal do Merge Sort . . . 25

10 Metodologia experimental **26**
10.1 Configurações comparadas . . . 26
10.2 Cargas e repetições . . . 26
10.3 Métricas previstas . . . 26
10.4 Reprodutibilidade . . . 26

11 Resultados e gráficos **26**

12 Discussão de escalabilidade **26**

13 Relação com IA generativa e RAG **27**

14 Limitações e ameaças à validade **27**

15 Declaração de uso de IA generativa **27**

16 Contribuição individual **28**

17 Vídeo da atividade **28**

18 Referências **28**

1. Identificação da equipe

A Atividade 2 (AV2) é desenvolvida por seis discentes do Programa de Pós-Graduação em Ciência da Computação (PROCC/UFS), na disciplina Projeto e Análise de Algoritmos, período 2026.2.

As responsabilidades foram distribuídas conforme as etapas previstas no enunciado, contemplando especificação formal, implementação, análise assintótica, experimentação e relação com inteligência artificial generativa.

Tabela 1: Distribuição de responsabilidades na AV2.

Integrante	**Frente principal**
João Cosme Sena Sá	Parte B: algoritmo e corretude; Parte D: experimentos e comparação.
Eduardo Henrique do Lago Silva	Parte B: algoritmo e corretude; Parte C: análise assintótica; Parte D: experimentos e comparação.
Helena Carvalho Leal	Parte D: experimentos, comparação e métricas; Parte E: relação com IA generativa.
Hernandison da Silva Bispo	Parte A: especificação do problema; Parte E: relação com IA generativa.
Nadianne Maria dos Santos Galvão	Parte A: especificação formal do problema; Parte C: análise assintótica.
Rafael Takeguma Goto	Parte B: algoritmo e corretude; Parte E: relação com IA generativa.

A divisão apresentada estabelece as responsabilidades principais de cada integrante, permanecendo a revisão e a validação dos resultados como atividades coletivas.

> A divisão acima é a **divisão de frentes** acordada (`docs/CONTRIBUICOES.md`); a revisão de código, da prova e dos resultados é responsabilidade coletiva. A tabela de contribuição individual **com evidências** é mantida em `docs/CONTRIBUICOES.md` e resumida na Seção 16.

2. Tema, corpus e fonte

O tema é o de número 8 do enunciado, **materiais educacionais abertos**. O corpus é o livro *Pense Python*, tradução livre da 3ª edição de *Think Python*, de Allen B. Downey, traduzida por Rodrigo Castelan Carlson — conteúdo didático público de introdução à programação em Python, sem qualquer dado sensível, pessoal ou sigiloso.

Campo	**Registro**
Nome	*Pense Python* — tradução livre da 3ª ed. de *Think Python*
Autor / Tradução	Allen B. Downey / Rodrigo Castelan Carlson
Fonte	`https://rodrigocarlson.github.io/PensePython3ed/` ; `https://github.com/rodrigocarlson/PensePython3ed`
Commit de referência	`cfde2c450bf4f2ea65326da88e9968f400597e92` (branch `main`)
Recorte	21 notebooks Jupyter: prefácio, introdução e capítulos 1 a 19 (`capitulos/*.ipynb`)
Tamanho auditado	817.578 bytes · 2.789 células · ≈73.867 palavras
SHA-256 do recorte	`2010626258...5768dc5` (agregado; ver `data/corpus_manifest.csv`)
Idioma	Português brasileiro (com estrangeirismos técnicos do Python/Jupyter)
Licença do texto / código	CC BY-NC-SA 4.0 / MIT
Data de acesso	02/09/2026 (aquisição registrada); planejamento em 01/09/2026
Dados sensíveis	Nenhum — conteúdo didático público

2.1 Continuidade da Atividade 1

A Atividade 2 (AV2) dá continuidade ao protótipo de recuperação de contexto desenvolvido na Atividade 1 (AV1), preservando o corpus *Pense Python* e os mecanismos de recuperação lexical.

Na AV1, o sistema recuperava e ordenava fragmentos textuais (*chunks*) a partir de consultas em português, utilizando pontuações de relevância baseadas em frequência de termos e frequência inversa de documento.

Na AV2, incorpora-se uma etapa de otimização responsável por selecionar um subconjunto de chunks que maximize a relevância estimada, respeitando um orçamento máximo de unidades lexicais.

O problema será modelado como mochila 0/1, utilizando programação dinâmica como abordagem principal e uma estratégia gulosa por relevância/token como linha de base, conforme a formulação clássica de otimização combinatória apresentada por Kleinberg e Tardos [2].

> **Corpus não versionado, mas reprodutível.** Como a licença do texto é não comercial e com compartilhamento pelas mesmas condições (CC BY-NC-SA), o texto não é redistribuído no repositório (cujo código é MIT). O script `scripts/baixar_corpus.py` baixa o material no commit fixado e `data/corpus_manifest.csv` guarda o SHA-256 de cada notebook e um **hash agregado do recorte**, permitindo conferir que qualquer download reproduz o mesmo conjunto auditado. Trechos citados preservam atribuição ao autor e ao tradutor.

Consultas de recuperação. As consultas seguem o formato "tira-dúvidas" de um estudante sobre o livro, por exemplo: *"como funciona um laço `for` em Python?"*, *"o que é uma função recursiva?"* e *"como tratar exceções com `try` e `except`?"*.

O conjunto de 30 consultas, definido na AV1 antes da observação dos resultados, será reutilizado na AV2, preservando-se o arquivo `data/queries.csv`.

Para cada consulta, os chunks recuperados constituirão o conjunto de candidatos submetido às estratégias de programação dinâmica e gulosa, sob o mesmo orçamento de tokens.

3. Repositório, licença, versão e data de acesso

Item	Registro
URL do repositório	`https://github.com/joaocss/PAA_UFS_2026_2_SenaSa_Joao_Bispo_Hernandison_Galvao_Nadianne_Goto_Rafael_Leal_Helena_Silva_Eduardo`
Licença do código	MIT (arquivo `LICENSE`)
Versão da AV2	Em desenvolvimento na branch `av2-programacao-dinamica`; tag ainda não publicada
Commit/tag/release da AV2	*A registrar na integração final, prevista para 04/11/2026*
Origem do corpus	Aquisição fixada pela AV1; data e commit verificáveis em `data/README.md`
Ambiente travado	`requirements.lock`: pytest 8.3.4, scikit-learn 1.6.1, matplotlib 3.10.0, numpy 2.2.2

O repositório não contém chaves, senhas, tokens, dados pessoais ou artefatos sem licença compatível. Arquivos grandes são controlados por `.gitignore`; as instruções de obtenção e reprodução estão no `README.md`.

4. Especificação do problema

4.1 Definição do problema

O problema investigado consiste em selecionar, entre os chunks recuperados pelo mecanismo lexical da AV1 para uma determinada consulta, um subconjunto que maximize a relevância estimada sem ultrapassar um orçamento máximo de unidades lexicais.

Cada chunk possui um custo, associado à sua quantidade de unidades lexicais, e um valor, determinado pela pontuação de relevância atribuída durante a recuperação.

A seleção é modelada como um problema da mochila 0/1 (*0/1 Knapsack Problem*), no qual cada chunk pode ser incluído integralmente ou excluído da solução, sem possibilidade de fracionamento.

Essa modelagem constitui uma adaptação do problema clássico de otimização combinatória apresentado por Kleinberg e Tardos [2].

4.2 Corpus e representação dos chunks

O corpus utilizado corresponde ao livro *Pense Python*, tradução brasileira da terceira edição de *Think Python*, de Allen B. Downey, realizada por Rodrigo Castelan Carlson [7].

O recorte, preservado da AV1, compreende 21 notebooks Jupyter, incluindo materiais introdutórios e os capítulos 1 a 19.

O arquivo de dados analisado contém 2.737 chunks, sendo 1.405 de texto Markdown e 1.332 de código. Cada chunk possui identificador, conteúdo textual, ordem de origem e metadados de localização no corpus.

Cada candidato c_i será representado pelos seguintes atributos:

- `chunk_id`: identificador único;
- `texto`: conteúdo normalizado;
- `n_tokens`: custo em unidades lexicais;
- `score_relevancia`: pontuação lexical calculada para a consulta;
- `source_order`: posição do chunk no corpus original;
- `origem`: metadados de procedência.

O campo `score_relevancia` é calculado durante a recuperação e não integra originalmente o arquivo de chunks da AV1.

O corpus processado da AV1 será preservado. Para reprodução da AV2, será utilizado o arquivo versionado `data/chunks.jsonl`, cuja equivalência ao arquivo gerado `data/processed/chunks.jsonl` deverá ser conferida pelos hashes documentados no repositório.

Os registros originais possuem o campo `n_tokens`, que será inicialmente utilizado como custo de seleção, expresso em unidades lexicais do texto normalizado.

O script de preparação utiliza, por padrão, tamanho máximo de 256 unidades e sobreposição configurada de 32 unidades entre fragmentos.

Essa contagem constitui uma aproximação do custo de contexto, não correspondendo necessariamente à quantidade de tokens utilizada por um modelo de linguagem específico.

A representação da AV2 preservará os identificadores e metadados originais, acrescentando a pontuação de relevância calculada para cada consulta.

Dessa forma, ambos os algoritmos receberão os mesmos candidatos, custos e valores.

4.3 Função de relevância

A relevância lexical é calculada pelo mecanismo TF-IDF desenvolvido na AV1, com base na frequência de termos e na frequência inversa de documento, conceitos apresentados por Manning, Raghavan e Schütze [5].

Na AV2, preserva-se a mesma função de pontuação, de modo que as diferenças entre os algoritmos comparados estejam relacionadas à seleção dos chunks, e não a alterações no cálculo da relevância.

A frequência ponderada de um termo é definida por:

$$\mathrm{tf}(t,c) = \begin{cases} 1 + \ln f_{t,c}, & \text{se } f_{t,c} > 0, \\ 0, & \text{caso contrário.} \end{cases} \tag{1}$$

A frequência inversa de documento é calculada por:

$$\mathrm{idf}(t) = \ln\left(\frac{N+1}{\mathrm{df}(t)+1}\right) + 1, \tag{2}$$

em que N representa a quantidade total de chunks do corpus e $\mathrm{df}(t)$ corresponde ao número de chunks que contêm o termo t.

A pontuação de relevância para uma consulta q e um chunk c é:

$$\mathrm{score}(q,c) = \sum_{t \in \mathrm{TermosDistintos}(q)} qtf_t \cdot tf(t,c) \cdot idf(t). \tag{3}$$

A soma considera os termos distintos da consulta, enquanto qtf_t representa a frequência de ocorrência de cada termo na consulta.

A formulação preserva o mecanismo de relevância implementado na AV1, mantendo a mesma pontuação para os algoritmos comparados.

Na AV2, essa pontuação será utilizada como valor do item no problema da mochila 0/1:

$$v_i = \mathrm{score}(q, c_i). \tag{4}$$

Dessa forma, a programação dinâmica e a estratégia gulosa utilizarão os mesmos valores de relevância para cada candidato.

Na recuperação da AV1, os chunks são ordenados por pontuação decrescente e, em caso de empate, pelo menor `source_order`. Essa regra será preservada na recuperação dos candidatos.

Ressalta-se que o TF-IDF constitui uma estimativa de relevância lexical. Portanto, a soma dos scores não representa necessariamente maior cobertura semântica, diversidade de informações ou ausência de redundância entre os chunks.

4.4 Definição formal da entrada

Para uma consulta q, considera-se o conjunto de chunks candidatos recuperados pelo mecanismo lexical da AV1, com pontuação de relevância positiva:

$$C_q = \{c_1, c_2, \ldots, c_n\}, \tag{5}$$

em que $n = |C_q|$ representa a quantidade de candidatos disponíveis para a seleção, e não o número total de chunks do corpus.

Cada candidato c_i possui:

- um custo $w_i \in \mathbb{N}^+$, correspondente à quantidade de unidades lexicais registrada em `n_tokens`;
- um valor $v_i \in \mathbb{R}_{\geq 0}$, correspondente à pontuação de relevância estimada para a consulta q.

Define-se $W \in \mathbb{N}_0$ como o orçamento máximo de unidades lexicais disponível para a seleção.

Assim, a entrada do problema é representada por:

$$I = (q, C_q, W), \tag{6}$$

em que cada candidato c_i está associado ao par (w_i, v_i).

Essa representação adapta a formulação clássica do problema da mochila 0/1, apresentada por Kleinberg e Tardos [2].

4.5 Definição formal da saída

A saída consiste em um subconjunto de chunks selecionados:

$$S \subseteq C_q, \tag{7}$$

no qual cada chunk pode ser incluído integralmente ou excluído, sem possibilidade de fracionamento ou seleção repetida. Uma solução é considerada viável quando respeita o orçamento máximo de unidades lexicais:

$$\sum_{c_i \in S} w_i \leq W. \tag{8}$$

Além do subconjunto selecionado, serão registrados o custo total da seleção e a relevância acumulada, definidos, respectivamente, por:

$$W_S = \sum_{c_i \in S} w_i, \tag{9}$$

$$V_S = \sum_{c_i \in S} v_i. \tag{10}$$

A saída deverá preservar os identificadores dos chunks selecionados, permitindo recuperar seus textos e metadados de origem. A exigência de maximizar a relevância acumulada será formalizada na função objetivo.

4.6 Função objetivo

O objetivo da seleção é maximizar a relevância lexical acumulada dos chunks escolhidos, considerando o conjunto de candidatos recuperados para uma consulta.

Para cada candidato c_i, define-se uma variável binária de decisão:

$$x_i = \begin{cases} 1, & \text{se o chunk } c_i \text{ for selecionado,} \\ 0, & \text{caso contrário.} \end{cases} \tag{11}$$

A função objetivo é definida por:

$$\max V_S = \max \sum_{i=1}^{n} v_i x_i. \tag{12}$$

Nessa expressão, n representa a quantidade de candidatos, v_i corresponde à relevância lexical do chunk c_i para a consulta e x_i indica sua inclusão na seleção.

A formulação segue o problema clássico da mochila 0/1, no qual se busca maximizar o valor total dos itens selecionados sob uma restrição de capacidade [2].

Neste trabalho, o valor de cada item corresponde ao score lexical calculado pelo mecanismo TF-IDF da AV1. Portanto, a maximização refere-se à relevância estimada pelo modelo adotado.

A função objetivo pressupõe valores aditivos: a relevância total é calculada pela soma dos scores individuais dos chunks selecionados.

Entretanto, essa soma não considera diretamente a redundância, as dependências ou a diversidade de informações entre os fragmentos.

Consequentemente, a solução ótima da mochila 0/1 não garante, isoladamente, maior cobertura semântica ou melhor qualidade das respostas geradas por um modelo de linguagem.

4.7 Restrições do problema

A seleção de contexto está sujeita a uma restrição de capacidade, determinada pelo orçamento máximo de unidades lexicais W.

A soma dos custos dos chunks selecionados não pode ultrapassar esse limite:

$$\sum_{i=1}^{n} w_i x_i \leq W, \tag{13}$$

em que w_i representa o custo do chunk c_i, obtido do campo `n_tokens` da AV1, e x_i indica sua seleção. Adicionalmente, cada candidato está sujeito à restrição de integralidade:

$$x_i \in \{0, 1\}, \quad i = 1, \ldots, n. \tag{14}$$

Assim, um chunk deve ser selecionado integralmente ou excluído, não podendo ser fracionado nem incluído mais de uma vez.

Considera-se também que os custos são inteiros positivos ($w_i \in \mathbb{N}^+$), os valores de relevância são reais não negativos e finitos ($v_i \in \mathbb{R}_{\geq 0}$), e o orçamento é um inteiro não negativo ($W \in \mathbb{N}_0$).

Essas condições caracterizam a formulação clássica da mochila 0/1, adaptada à seleção de contexto [2]. O orçamento W e os custos w_i serão expressos na mesma unidade, garantindo a consistência da restrição de capacidade.

4.8 Critério de corretude

Uma solução produzida pela programação dinâmica será considerada correta quando satisfizer simultaneamente duas propriedades: viabilidade e otimalidade.

A viabilidade exige que todos os chunks selecionados pertençam ao conjunto de candidatos C_q, que cada chunk seja incluído no máximo uma vez e que o orçamento de unidades lexicais seja respeitado:

$$S \subseteq C_q \quad \text{e} \quad \sum_{c_i \in S} w_i \leq W. \tag{15}$$

Define-se o conjunto de soluções viáveis por:

$$\mathcal{F} = \left\{ S' \subseteq C_q \;\middle|\; \sum_{c_i \in S'} w_i \leq W \right\}. \tag{16}$$

A otimalidade exige que não exista outro subconjunto viável com relevância acumulada superior à solução encontrada. Formalmente, uma solução ótima S^* deve satisfazer:

$$\sum_{c_i \in S^*} v_i \geq \sum_{c_i \in S'} v_i, \quad \forall S' \in \mathcal{F}. \tag{17}$$

Esse critério corresponde à otimalidade do problema clássico da mochila 0/1, fundamentado na programação dinâmica [2].

A programação dinâmica deverá satisfazer as condições de viabilidade e otimalidade. Por sua vez, a estratégia gulosa deverá garantir a viabilidade, mas não necessariamente a otimalidade global.

Ressalta-se que a corretude é estabelecida em relação à função objetivo e às restrições matemáticas definidas.

Portanto, uma solução ótima não garante, isoladamente, relevância semântica perfeita, ausência de redundância ou maior qualidade das respostas produzidas por um modelo de linguagem.

A demonstração formal da corretude do algoritmo implementado será apresentada na Parte B.

4.9 Linha de base: estratégia gulosa

Como linha de base (*baseline*), será utilizada uma estratégia gulosa que prioriza os chunks pela razão entre a relevância lexical estimada e o custo em unidades lexicais. Para cada candidato c_i, define-se:

$$r_i = \frac{v_i}{w_i}, \tag{18}$$

em que v_i representa a pontuação de relevância do candidato e w_i corresponde ao custo obtido do campo `n_tokens` da AV1.

Os candidatos serão considerados em ordem decrescente de r_i. Inicialmente, o orçamento restante será igual a W.

Cada chunk será selecionado integralmente caso seu custo não ultrapasse o orçamento restante. Após sua inclusão, o custo correspondente será descontado do orçamento disponível. Caso o candidato não caiba, será ignorado, e a avaliação continuará com o próximo item.

Em caso de empate na razão, será priorizado o candidato com maior relevância absoluta e, persistindo o empate, aquele com menor `source_order`, garantindo uma ordenação determinística.

Embora a estratégia gulosa apresente uma regra de escolha simples, ela não garante a solução ótima para a mochila 0/1, pois escolhas locais podem impedir combinações globalmente superiores [2]. A programação dinâmica e o baseline guloso receberão os mesmos candidatos, custos, pontuações de relevância e orçamento W.

A comparação considerará a relevância acumulada V_S, o custo total W_S, o tempo de seleção e o consumo aproximado de memória. A avaliação experimental dessas métricas será realizada posteriormente, na Parte D.

4.10 Casos de borda

Para assegurar a consistência da seleção de contexto, foram definidos casos de borda que deverão ser considerados na validação dos algoritmos.

A Tabela 2 apresenta os cenários e os comportamentos esperados.

Tabela 2: Casos de borda da seleção de contexto.

Caso	**Comportamento esperado**
Candidatos vazios	Se $C_q = \varnothing$, retornar $S = \varnothing$, com custo e relevância total iguais a zero.
Consulta sem correspondência	Caso nenhum chunk apresente relevância positiva, retornar o conjunto vazio.
Orçamento nulo	Se $W = 0$, retornar $S = \varnothing$, considerando que todos os custos são positivos.
Nenhum chunk cabe	Se $w_i > W$ para todos os candidatos, retornar o conjunto vazio.
Todos os chunks cabem	Se $\sum_{i=1}^{n} w_i \leq W$, selecionar todos os candidatos com relevância positiva.
Chunk maior que o orçamento	Candidatos com $w_i > W$ não podem integrar a solução, mas os demais continuam elegíveis.
Chunks duplicados	Identificadores repetidos serão tratados como entrada inválida. Chunks com textos idênticos, mas identificadores distintos, serão preservados inicialmente, e sua possível redundância será considerada na avaliação experimental.
Empate de relevância	Quando existirem soluções com o mesmo valor ótimo, priorizar aquela com menor custo total e, persistindo o empate, a menor sequência lexicográfica dos valores de `source_order` selecionados.
Empate na razão relevância/custo	No baseline guloso, priorizar maior relevância absoluta e, persistindo o empate, menor `source_order`.
Relevância zero	Chunks com $v_i = 0$ não acrescentam valor à função objetivo e serão descartados antes da seleção.

Os comportamentos descritos constituem requisitos esperados da implementação, e não resultados já observados.

Sua verificação será realizada posteriormente, por meio de testes automatizados e experimentos específicos na Parte D.

Para a programação dinâmica, o desempate entre soluções ótimas seguirá, nesta ordem, a maior relevância acumulada, o menor custo total e a menor sequência lexicográfica dos valores de `source_order`,

previamente ordenados em ordem crescente.

4.11 Parâmetros experimentais preliminares

Os experimentos serão planejados para comparar a programação dinâmica e a estratégia gulosa sob condições equivalentes, reutilizando o corpus, as consultas e o mecanismo de recuperação da AV1.

A Tabela 3 apresenta os parâmetros experimentais preliminares.

Tabela 3: Parâmetros experimentais preliminares da AV2.

Parâmetro	**Definição preliminar**
Corpus	2.737 chunks do livro *Pense Python*, reutilizados da AV1.
Consultas	30 consultas previamente definidas em `data/queries.csv`.
Recuperação	Configuração C3 da AV1: busca linear com ordenação por Merge Sort.
Orçamento (W)	256, 512 e 1.024 unidades lexicais, sujeitos à validação.
Quantidade de candidatos (n)	10, 30 e 50 candidatos por consulta, limitados aos resultados com relevância positiva.
Repetições	No mínimo três execuções por cenário.
Algoritmo principal	Mochila 0/1 com programação dinâmica.
Linha de base	Estratégia gulosa por relevância/custo.
Métricas de qualidade	Relevância acumulada (V_S), custo total (W_S), taxa de utilização do orçamento, redundância e diferença de relevância entre as estratégias.
Métricas computacionais	Tempo de seleção, tempo total do fluxo e consumo aproximado de memória.

Para assegurar a comparabilidade, ambos os algoritmos receberão os mesmos candidatos, custos, pontuações de relevância e orçamento.

O mecanismo de recuperação permanecerá fixo durante as comparações, de modo que as diferenças observadas possam ser atribuídas às estratégias de seleção. Os valores apresentados constituem um planejamento inicial. A quantidade efetiva de candidatos, os orçamentos e as repetições serão confirmados antes da execução dos experimentos.

O custo dos chunks será inicialmente obtido do campo `n_tokens`, interpretado como contagem de unidades lexicais, e não como tokenização exata de um LLM. A implementação dos algoritmos, a execução dos experimentos, a coleta das métricas e a análise dos resultados serão realizadas nas Partes B e D, respectivamente.

4.12 Decisões metodológicas

Para preservar a continuidade da AV1, será utilizada a configuração C3, composta por busca linear e Merge Sort, como mecanismo fixo de recuperação dos candidatos.

O custo w_i será definido inicialmente pelo campo `n_tokens` da AV1, produzido pela fragmentação com a expressão regular `\w+` aplicada ao texto normalizado. A contagem representa unidades lexicais, não necessariamente tokens de um modelo de linguagem. A normalização utilizada para calcular a pontuação lexical difere daquela utilizada na fragmentação; portanto, valor e custo não são contagens da mesma representação.

Em caso de múltiplas soluções com a mesma relevância acumulada máxima na programação dinâmica, será priorizada aquela com menor custo total.

Persistindo o empate, será selecionado o subconjunto cuja sequência crescente de valores `source_order` seja lexicograficamente menor, garantindo uma regra determinística. A programação dinâmica e

a estratégia gulosa receberão os mesmos conjuntos de candidatos, valores de relevância, custos e orçamentos.

Essa padronização busca assegurar que as diferenças observadas nos experimentos sejam atribuídas às estratégias de seleção, e não a alterações nos dados de entrada.

5. Representação dos dados e estratégia de chunking

5.1 Normalização

A AV2 preserva o procedimento de normalização desenvolvido na AV1, mantendo as seguintes regras:

- **Células Markdown:** normalização NFKC, conversão para minúsculas, separação controlada da pontuação e compactação de espaços, preservando os acentos.
- **Células de código:** preservação do código-fonte original, incluindo a indentação, com representação lexical separada.
- **Stopwords:** não são removidas na configuração principal, preservando termos funcionais relevantes para as consultas.
- **Saídas de execução:** são descartadas durante a preparação do corpus.

5.2 Segmentação (chunking)

A segmentação preserva a hierarquia do livro, considerando notebook, capítulo e seção, sem combinar células Markdown e código em um mesmo chunk.

O procedimento de preparação da AV1 utiliza, por padrão, tamanho máximo de 256 unidades lexicais e sobreposição configurada de 32 unidades, produzindo 2.737 chunks.

As configurações 128/16 e 512/64 foram consideradas na análise de sensibilidade da AV1, não sendo inicialmente adotadas como configurações principais da AV2.

A contagem registrada no campo `n_tokens` será utilizada inicialmente como aproximação do custo de seleção. Essa medida não equivale necessariamente à tokenização de um modelo de linguagem.

5.3 Representações computacionais

A AV2 reutiliza a representação lexical e os mecanismos de pontuação desenvolvidos na AV1, incorporando uma estrutura específica para a seleção de contexto.

- **TF-IDF:** representa a relevância lexical dos chunks em relação à consulta, fornecendo o valor v_i de cada candidato.
- **Custo de seleção:** cada chunk possui um custo w_i, associado à contagem de unidades lexicais registrada no corpus.
- **Representação da mochila 0/1:** cada candidato é representado pelo par (w_i, v_i), acompanhado de seu identificador e metadados de origem.
- **Orçamento:** o limite W estabelece a capacidade máxima disponível para selecionar os chunks.

A recuperação dos candidatos será realizada pela configuração C3 da AV1, composta por busca linear e Merge Sort, mantendo fixos os critérios de pontuação e ordenação.

> **Limitações da representação.**
>
> O corpus contém termos técnicos em inglês, como *loop* e *string*, que podem influenciar a relevância lexical.
>
> Além disso, a utilização de um único livro introdutório limita a generalização dos resultados para outros materiais educacionais.
>
> A função objetivo considera a soma das relevâncias individuais, não modelando diretamente redundância ou cobertura semântica entre os chunks.

6. Algoritmos e pseudocódigo

A AV2 reutiliza o mecanismo de recuperação lexical da AV1, adotando a configuração C3, composta por busca linear e Merge Sort.

A principal alteração consiste na inclusão de uma etapa de seleção de contexto sob restrição de unidades lexicais, modelada como mochila 0/1.

Serão consideradas duas abordagens: programação dinâmica, como algoritmo principal, e uma estratégia gulosa por relevância/custo, utilizada como linha de base.

6.1 Programação dinâmica: mochila 0/1

6.2 Estratégia gulosa por relevância/custo

6.3 Justificativa de corretude

7. Análise no modelo RAM

7.1 Parâmetros

A análise assintótica considera a implementação dos algoritmos de seleção de contexto desenvolvida na Parte B, distinguindo os custos da recuperação lexical daqueles associados à seleção dos chunks.

A Tabela 4 apresenta os parâmetros utilizados na análise.

O parâmetro N representa o tamanho do corpus utilizado pela recuperação lexical da AV1, enquanto n corresponde à quantidade de candidatos efetivamente fornecidos aos algoritmos de seleção da AV2.

Essa distinção permite analisar separadamente o custo da recuperação e o custo da seleção.

Na programação dinâmica, o orçamento W também influencia diretamente a quantidade de estados processados. Na estratégia gulosa, o custo computacional depende principalmente da quantidade de candidatos e de sua ordenação.

As análises seguintes adotam inicialmente o modelo RAM com operações elementares de custo constante, explicitando posteriormente as particularidades da representação numérica empregada na implementação.

Tabela 4: Parâmetros da análise de complexidade.

Símbolo	Significado
N	Número total de chunks do corpus.
n	Número de candidatos submetidos à seleção.
W	Orçamento máximo em unidades lexicais.
w_i	Custo lexical do candidato i.
v_i	Valor de relevância lexical do candidato i.
m	Número de termos da consulta.
k	Número de chunks efetivamente selecionados, com $0 \leq k \leq n$.
r	Número de repetições experimentais.
Q	Comprimento da consulta em caracteres.
L	Soma dos comprimentos dos textos dos N chunks, em caracteres.
P	Número de resultados com relevância positiva submetidos ao Merge Sort, com $0 \leq P \leq N$.

7.2 Operações elementares

A análise de complexidade considera o modelo RAM (*Random Access Machine*), no qual atribuições, comparações, operações aritméticas sobre valores de tamanho limitado e acessos a posições de memória possuem custo constante.

A análise utiliza como referência a implementação da Parte B, disponível no módulo `src/paa_context/selecao.p` considerando as etapas de validação, ordenação, seleção e construção da saída.

Validação dos candidatos. Ambas as estratégias utilizam a função `_validar()`, responsável por verificar os dados de entrada, incluindo custos positivos, valores válidos, orçamento não negativo e unicidade dos identificadores e dos valores de `source_order`.

A validação percorre os n candidatos uma vez, realizando um número limitado de verificações por candidato.

A detecção de duplicatas utiliza conjuntos hash, com operações de consulta e inserção de custo esperado constante.

Assim, a validação apresenta custo esperado $O(n)$, sob as hipóteses adotadas.

Ordenação dos candidatos. A programação dinâmica ordena os candidatos pelo campo `source_order`, enquanto o algoritmo guloso utiliza a razão entre relevância e custo lexical.

Ambas as estratégias utilizam a função `sorted()` do Python, cujo algoritmo de ordenação apresenta limite superior $O(n \log n)$.

No guloso, a prioridade é construída com a classe `Fraction`, utilizando os valores inteiros quantizados da Parte B.

A análise RAM inicial considera custo constante para operações aritméticas sobre valores de tamanho limitado.

Preenchimento da programação dinâmica. A função `mochila_dp()` constrói uma matriz bidimensional E com $(n+1)(W+1)$ posições.

O preenchimento utiliza dois laços aninhados: o primeiro percorre os n candidatos, e o segundo considera as capacidades de 0 até W.

Portanto, a quantidade de estados processados é:

$$Q_{\mathrm{DP}}(n, W) = n(W + 1). \tag{19}$$

Em cada estado, são realizados acessos à matriz, atribuições, comparações e operações aritméticas de custo constante no modelo adotado.

A quantidade de estados processados não deve ser confundida com o número de posições alocadas na matriz, que corresponde a $(n + 1)(W + 1)$.

Reconstrução da solução. Após o preenchimento da matriz, a programação dinâmica percorre os n candidatos para reconstruir o subconjunto selecionado.

Esse percurso realiza até n verificações, com custo $O(n)$.

Posteriormente, os k chunks escolhidos, com $0 \leq k \leq n$, são ordenados conforme sua posição no ranking original da AV1.

Essa ordenação possui limite superior $O(k \log k)$.

Seleção gulosa. Após a ordenação por relevância/custo, a função `_guloso()` percorre os n candidatos.

Para cada candidato, verifica-se se seu custo não ultrapassa o orçamento restante.

Quando o candidato cabe, ele é selecionado integralmente e seu custo é descontado do orçamento disponível.

Caso contrário, o algoritmo continua avaliando os candidatos seguintes.

Assim, o percurso de seleção realiza exatamente n iterações, apresentando custo $\Theta(n)$ sob o modelo RAM adotado.

As etapas identificadas fundamentam a análise das complexidades temporais e espaciais apresentadas nas subseções seguintes.

7.3 Programação dinâmica: mochila 0/1

A análise considera a implementação `mochila_dp()`, desenvolvida na Parte B, que utiliza programação dinâmica iterativa (*bottom-up*) para selecionar chunks sob restrição de orçamento.

A função recebe n candidatos e um orçamento máximo W, expresso em unidades lexicais.

Complexidade temporal. Inicialmente, os candidatos são validados e ordenados pelo campo `source_order`.

A validação apresenta custo esperado $O(n)$, considerando custo esperado constante para consultas e inserções em conjuntos hash. A ordenação possui limite superior $O(n \log n)$.

Em seguida, a implementação inicializa uma matriz bidimensional E com $(n + 1)(W + 1)$ posições.

O preenchimento é realizado por dois laços aninhados, percorrendo os n candidatos e as $W + 1$ capacidades possíveis.

Desse modo, são processados exatamente $n(W+1)$ estados, com custo constante por estado sob as hipóteses do modelo RAM.

Após o preenchimento, a solução é reconstruída por um percurso de n candidatos, seguido da ordenação dos k chunks selecionados segundo sua posição no ranking da AV1, com $k \leq n$.

Portanto, o custo temporal total pode ser expresso por:

$$\begin{aligned} T_{\mathrm{DP}}(n, W) = {} & O(n) + O(n \log n) \\ & + \Theta((n+1)(W+1)) \\ & + O(n) + O(k \log k). \end{aligned} \tag{20}$$

Como $k \leq n$, e considerando o limite assintótico das operações de ordenação, obtém-se, para entradas válidas:

$$T_{\mathrm{DP}}(n, W) = \Theta\left((n+1)(W+1) + n \log n\right). \tag{21}$$

Para $n \geq 1$ e $W \geq 1$, essa expressão pode ser simplificada para:

$$T_{\mathrm{DP}}(n, W) = \Theta(nW + n \log n). \tag{22}$$

Quando W cresce de modo que $W = \Omega(\log n)$, o custo de preenchimento da matriz domina assintoticamente a ordenação, resultando em $\Theta(nW)$.

Complexidade espacial. A implementação armazena uma matriz bidimensional com $(n+1)(W+1)$ posições.

Adicionalmente, são utilizadas listas para ordenação, reconstrução e armazenamento dos chunks selecionados, com espaço auxiliar limitado por $O(n)$.

Logo, a complexidade espacial é:

$$M_{\mathrm{DP}}(n, W) = \Theta\left((n+1)(W+1) + n\right). \tag{23}$$

Como o espaço da matriz domina as demais estruturas, obtém-se:

$$M_{\mathrm{DP}}(n, W) = \Theta\left((n+1)(W+1)\right). \tag{24}$$

Natureza pseudopolinomial. A programação dinâmica apresenta complexidade pseudopolinomial, pois o número de estados depende do valor numérico do orçamento W, e não apenas da quantidade de bits necessária para representá-lo.

Assim, embora a implementação seja eficiente para valores moderados de W, orçamentos elevados podem aumentar significativamente o tempo de processamento e o consumo de memória.

Particularidades da implementação. A função utiliza valores de relevância quantizados em inteiros, conforme a representação adotada na Parte B.

As operações aritméticas são consideradas de custo constante na análise RAM inicial. Essa hipótese pressupõe valores de tamanho limitado; o custo de manipulação de inteiros arbitrariamente grandes não é detalhado neste modelo.

A otimalidade garantida pela implementação refere-se aos valores quantizados utilizados na função objetivo computacional, distinguindo-se da pontuação TF-IDF originalmente calculada pela AV1.

Verificação computacional do crescimento teórico. Como complemento à análise assintótica, foi desenvolvido um script em Python para calcular a quantidade de estados processados pela programação dinâmica, considerando diferentes quantidades de candidatos e orçamentos.

Foram utilizadas 15 configurações, com $n \in \{10, 20, 30, 40, 50\}$ e $W \in \{256, 512, 1024\}$.

A Figura 1 apresenta o crescimento do número de estados em função da quantidade de candidatos, mantendo-se o orçamento constante.

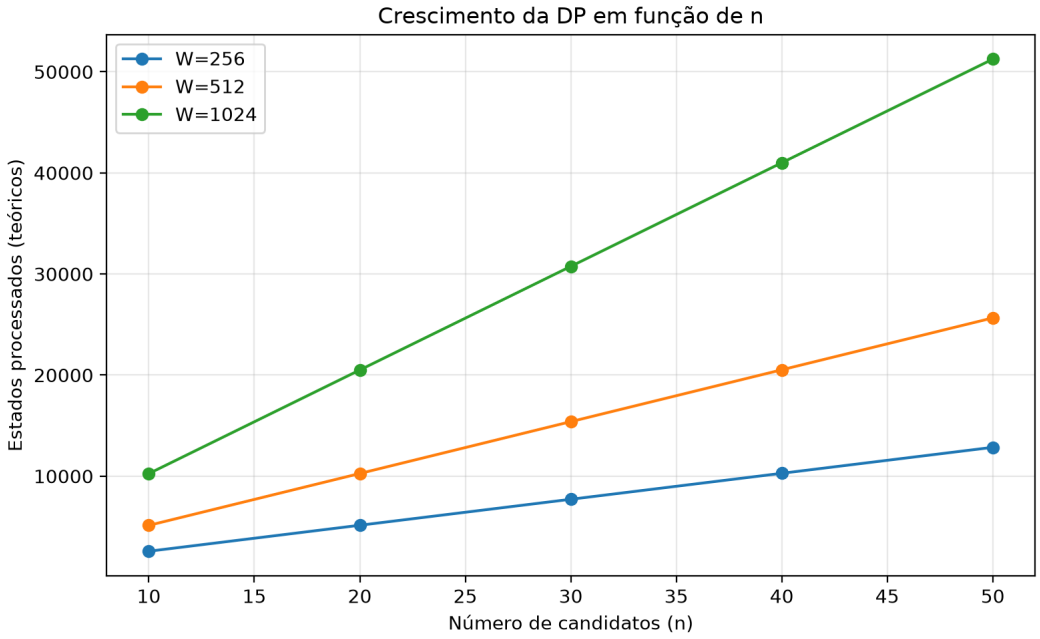


Figura 1: Crescimento teórico dos estados processados pela programação dinâmica em função de n, para diferentes orçamentos W.

A Figura 2 apresenta a variação dos estados em função do orçamento, mantendo-se fixa a quantidade de candidatos.

Os resultados são consistentes com a expressão $Q(n, W) = n(W + 1)$, evidenciando crescimento linear em cada parâmetro quando o outro permanece constante.

Ressalta-se que as figuras representam quantidades teóricas derivadas da estrutura do algoritmo, não constituindo medições empíricas de tempo de execução ou consumo de memória.

7.4 Estratégia gulosa por relevância/custo

A análise considera a implementação `guloso_densidade()`, que utiliza a função auxiliar `_guloso()` para selecionar candidatos segundo uma prioridade baseada na razão entre relevância e custo lexical.

Para cada candidato c_i, a prioridade principal é definida por:

$$r_i = \frac{v_i}{w_i}, \tag{25}$$

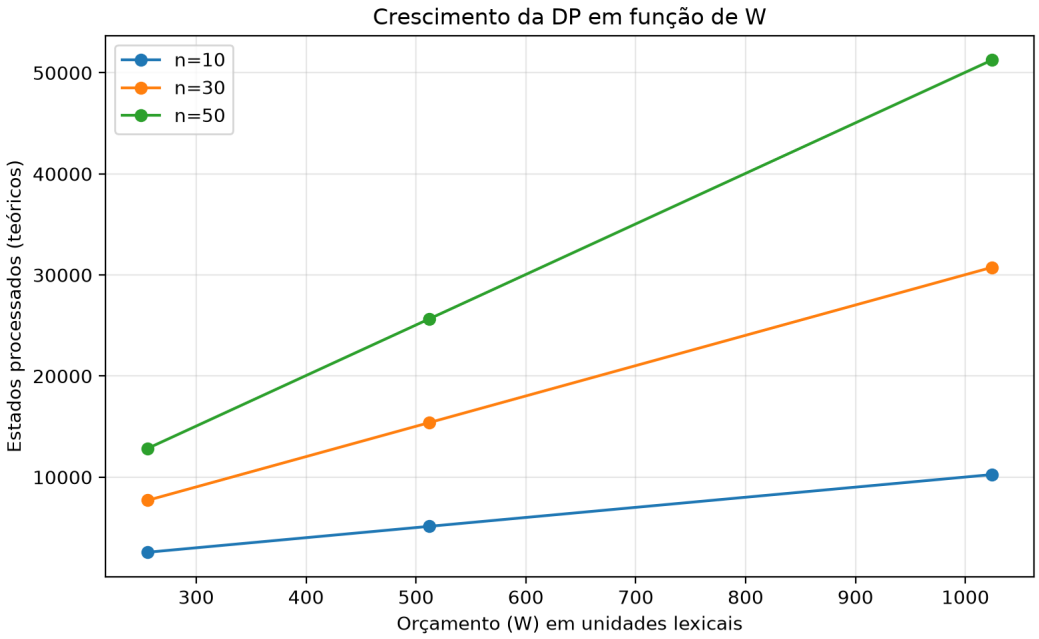


Figura 2: Crescimento teórico dos estados processados pela programação dinâmica em função de W, para diferentes quantidades de candidatos n.

em que v_i representa a relevância do candidato e w_i corresponde ao seu custo em unidades lexicais.

A implementação utiliza os valores inteiros quantizados da Parte B para calcular essa razão.

Em caso de empate, prioriza-se o candidato de maior valor absoluto e, persistindo o empate, aquele com menor `source_order`.

Complexidade temporal. Inicialmente, a função auxiliar `_validar()` percorre os n candidatos, verificando as pré-condições da entrada.

Considerando custo esperado constante para as operações sobre conjuntos hash, essa etapa apresenta custo esperado $O(n)$.

Em seguida, os candidatos são ordenados pela prioridade definida.

A implementação utiliza a função `sorted()` do Python, que emprega o algoritmo Timsort.

A construção das chaves de ordenação ocorre uma vez por candidato, enquanto a ordenação apresenta limite superior $O(n \log n)$.

Após a ordenação, um único laço percorre os n candidatos.

Para cada candidato, verifica-se se seu custo cabe no orçamento restante. Quando a condição é satisfeita, o candidato é selecionado e o orçamento disponível é atualizado.

Esse percurso realiza exatamente n iterações, apresentando custo $\Theta(n)$ no modelo RAM adotado.

Por fim, os k chunks selecionados, com $0 \leq k \leq n$, são ordenados conforme sua posição no ranking original da AV1.

Essa etapa apresenta limite $O(k \log k)$.

Assim, o custo temporal da implementação pode ser expresso por:

$$T_{\mathrm{G}}(n,k) = O(n) + O(n \log n) \\ + \Theta(n) + O(k \log k). \tag{26}$$

Como $k \leq n$, o limite superior da complexidade temporal é:

$$T_{\mathrm{G}}(n) = O(n \log n). \tag{27}$$

No pior caso, a ordenação pode demandar $\Theta(n \log n)$ comparações, caracterizando o comportamento assintótico predominante.

Em entradas favoráveis, a ordenação Timsort pode explorar sequências previamente ordenadas, reduzindo o número de operações realizadas.

Complexidade espacial. Diferentemente da programação dinâmica, a estratégia gulosa não constrói uma tabela de estados indexada pelo orçamento W.

A implementação utiliza conjuntos para validação, uma lista ordenada de candidatos e uma lista com os k chunks selecionados.

Como $k \leq n$, o espaço auxiliar é limitado por:

$$M_{\mathrm{G}}(n) = \Theta(n). \tag{28}$$

Esse resultado considera as estruturas auxiliares utilizadas na implementação, sem contabilizar a memória dos textos originais dos chunks.

Particularidades da implementação. A função `guloso_densidade()` utiliza a classe `Fraction` para representar exatamente a razão entre os valores inteiros quantizados e os custos lexicais.

Essa representação evita erros adicionais de arredondamento na comparação das razões.

A análise assintótica apresentada considera custo constante para operações aritméticas sobre valores de tamanho limitado, conforme a hipótese inicial do modelo RAM.

Em uma análise baseada no custo em bits, a manipulação de inteiros arbitrariamente grandes pode introduzir custos adicionais não representados pelas expressões anteriores.

Por fim, embora a estratégia gulosa produza soluções viáveis, ela não garante a maximização global da função objetivo da mochila 0/1.

7.5 Custo do pipeline de recuperação e seleção

A AV2 mantém o mecanismo de recuperação lexical desenvolvido na AV1, adotando a configuração C3, composta por busca linear e ordenação por Merge Sort.

A principal modificação introduzida na AV2 consiste na inclusão de uma etapa de seleção de contexto sob restrição de orçamento, executada após a recuperação dos candidatos.

Dessa forma, o custo computacional do pipeline pode ser decomposto em duas parcelas:

$$T_{\mathrm{total}} = T_{\mathrm{recuperacao}} + T_{\mathrm{selecao}}. \tag{29}$$

Custo da recuperação lexical. A configuração C3 da AV1 utiliza busca linear seguida de Merge Sort e seleção dos primeiros resultados do ranking.

Sejam N o número total de chunks do corpus, Q o comprimento da consulta em caracteres, L a soma dos comprimentos dos textos dos chunks e P a quantidade de resultados com pontuação de relevância positiva.

A função `linear_search()` percorre os N chunks, chamando `relevance_score()` para cada um deles.

Na implementação examinada, cada chamada normaliza novamente a consulta e o texto do chunk, constrói contadores de frequência e percorre os termos distintos da consulta para calcular a pontuação TF-IDF.

Sob o modelo RAM, considerando operações esperadas constantes sobre estruturas hash e o custo linear da normalização dos textos, obtém-se:

$$T_{\text{busca}}(N, Q, L) = \Theta(NQ + L). \tag{30}$$

A busca armazena apenas resultados com pontuação positiva, formando uma lista de P candidatos, com $0 \leq P \leq N$.

Essa lista é ordenada pela implementação recursiva de Merge Sort, sem atalhos para sequências previamente ordenadas.

Consequentemente, a ordenação apresenta:

$$T_{\text{merge}}(P) = \Theta(P \log(P + 1)). \tag{31}$$

Após a ordenação, a função `top_k()` percorre a lista de resultados, realiza uma filtragem adicional e retorna até n candidatos, com $n \leq P$.

Para valores positivos de k, essa etapa apresenta custo $O(P + n) = O(P)$.

Portanto, o custo total da recuperação C3, considerando busca, pontuação, ordenação e corte dos candidatos, é:

$$T_{\text{C3}}(N, Q, L, P) = \Theta\left(NQ + L + P \log(P + 1)\right). \tag{32}$$

A construção das estatísticas de frequência documental, realizada pela função `build_statistics()`, ocorre uma vez antes do processamento das consultas.

Quando incluída no custo de preparação do corpus, essa etapa acrescenta $\Theta(L)$, sob as mesmas hipóteses de processamento lexical.

Essa análise considera os comprimentos dos textos em caracteres, distinguindo-os do orçamento W, que utiliza as unidades lexicais registradas nos chunks.

Verificação computacional do Merge Sort. Como complemento à análise no modelo RAM, foi executada a implementação de Merge Sort da configuração C3 utilizando entradas sintéticas e determinísticas.

Foram considerados conjuntos contendo $P \in \{10, 30, 50, 100, 200\}$ candidatos.

O contador de comparações da implementação registrou, respectivamente, 23, 112, 232, 559 e 1296 comparações.

A Figura 3 apresenta esses valores juntamente com a função de referência $P \log_2 P$.

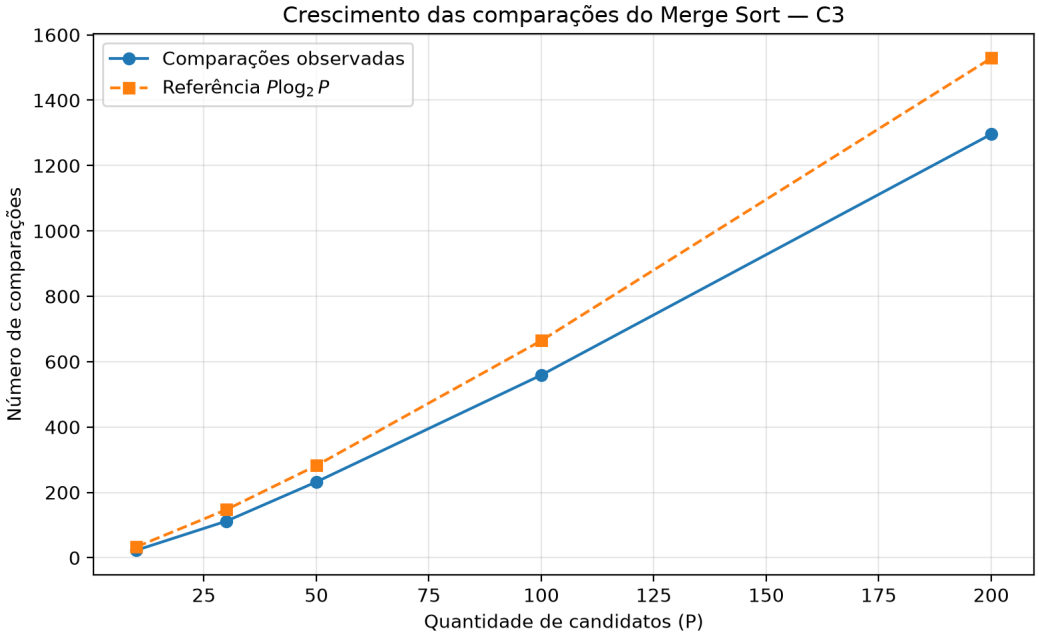


Figura 3: Comparações observadas no Merge Sort da configuração C3 e referência teórica $P \log_2 P$.

As contagens apresentaram crescimento compatível com a ordem assintótica $\Theta(P \log P)$, prevista pela recorrência do Merge Sort.

Ressalta-se que a função de referência não representa uma previsão exata do número de comparações, pois omite constantes multiplicativas e termos de menor ordem.

Os resultados correspondem a entradas sintéticas específicas e não constituem uma demonstração formal da complexidade, nem uma medição de tempo de execução.

Pipeline com programação dinâmica. Na programação dinâmica, a seleção dos n candidatos utiliza uma matriz indexada pelo orçamento W.

Conforme demonstrado anteriormente, para $n \geq 1$ e $W \geq 1$, seu custo é:

$$T_{\mathrm{DP}}(n, W) = \Theta(nW + n \log n). \tag{33}$$

Assim, o custo total pode ser expresso por:

$$T_{\mathrm{total,DP}}(N, Q, L, P, n, W) = \Theta\left(NQ + L + P \log(P+1) + nW + n \log n\right). \tag{34}$$

Pipeline com estratégia gulosa. A estratégia gulosa ordena os candidatos pela razão entre relevância e custo lexical, seguida de uma passagem de seleção.

Conforme a análise anterior, seu custo temporal apresenta limite superior:

$$T_{\mathrm{G}}(n) = O(n \log n). \tag{35}$$

Consequentemente, o custo total do pipeline com seleção gulosa é limitado por:

$$T_{\text{total,G}}(N, Q, L, P, n) = O\left(NQ + L + P\log(P+1) + n\log n\right). \tag{36}$$

As expressões incorporam o custo da recuperação lexical C3, considerando a normalização e pontuação dos N chunks, a ordenação dos P resultados positivos e a seleção de n candidatos.

A diferença entre os pipelines está na estratégia de seleção: a programação dinâmica possui custo dependente do orçamento W, enquanto a estratégia gulosa apresenta limite superior $O(n \log n)$.

A análise considera o modelo RAM, entradas válidas, $n \geq 1$, $W \geq 1$ e operações aritméticas de custo constante sobre valores de tamanho limitado.

Comparabilidade das estratégias. Nos experimentos da AV2, a etapa de recuperação lexical será mantida fixa, utilizando o mesmo corpus, as mesmas consultas e os mesmos critérios de pontuação e ordenação.

Dessa forma, a programação dinâmica e a estratégia gulosa receberão os mesmos n candidatos, valores, custos e orçamento.

A comparação isolada do tempo de seleção permitirá examinar o custo adicional introduzido por cada algoritmo.

Por sua vez, a medição do tempo total do pipeline permitirá avaliar a influência da seleção sobre o processamento completo.

Essa distinção é necessária porque um algoritmo de seleção assintoticamente mais custoso não implica, necessariamente, aumento proporcional no tempo total, especialmente quando a recuperação lexical representa uma parcela significativa do processamento.

As expressões apresentadas constituem previsões teóricas, cuja relação com os tempos observados será examinada na avaliação experimental da Parte D.

8. Melhor, pior e caso médio

A análise dos casos considera a implementação dos algoritmos desenvolvidos na Parte B, adotando o modelo RAM e as hipóteses de custo constante para operações elementares.

Programação dinâmica. A função `mochila_dp()` constrói uma matriz de $(n+1)(W+1)$ posições e realiza o preenchimento de $n(W+1)$ estados.

A quantidade de iterações não depende dos valores de relevância ou da quantidade de candidatos efetivamente selecionados.

Mesmo quando nenhum chunk cabe no orçamento, o preenchimento percorre todos os estados.

Assim, para n e W fixos, o custo do preenchimento é o mesmo assintoticamente nos melhores e piores casos:

$$T_{\text{preenchimento}}(n, W) = \Theta(n(W+1)). \tag{37}$$

Considerando também a validação, a ordenação inicial e a reconstrução da solução, obtém-se:

$$T_{\text{DP}}(n, W) = \Theta((n+1)(W+1) + n\log n) \tag{38}$$

como limite que contempla o custo das etapas da implementação.

A análise assume entradas válidas e execução completa do algoritmo.

Estratégia gulosa. A função `guloso_densidade()` realiza a validação dos candidatos, a ordenação por relevância/custo e um percurso linear de seleção.

O percurso examina os n candidatos, independentemente da quantidade efetivamente selecionada.

Entretanto, o custo da ordenação pode variar conforme a disposição inicial dos candidatos.

Como a implementação utiliza `sorted()` do Python, baseado em Timsort, o melhor caso de ordenação pode apresentar custo linear quando a entrada contém sequências favoráveis à ordenação.

No pior caso, o limite da ordenação é $O(n \log n)$.

Portanto, considerando também a ordenação final dos k chunks selecionados, o tempo total do guloso possui limite superior:

$$T_{\mathrm{G}}(n) = O(n \log n). \tag{39}$$

Em condições favoráveis às ordenações, o custo pode ser linear no número de candidatos.

Caso médio. A determinação rigorosa do caso médio depende da definição de uma distribuição de probabilidade sobre as entradas possíveis.

Como essa distribuição não foi estabelecida para os candidatos recuperados do corpus, não se atribui uma expressão probabilística específica ao caso médio.

Para a programação dinâmica, o número de estados processados permanece fixo para cada par (n, W), embora as operações condicionais possam variar.

No guloso, o comportamento da ordenação depende da estrutura das chaves e de sua disposição na entrada.

Dessa forma, os limites apresentados constituem garantias assintóticas, não estimativas probabilísticas de tempo médio de execução.

9. Recorrências

A análise das recorrências considera duas estruturas utilizadas no trabalho: a relação de estados da programação dinâmica para a mochila 0/1 e a recorrência temporal do Merge Sort empregado na recuperação C3.

Essas relações possuem finalidades distintas: a primeira define a solução do problema de otimização, enquanto a segunda descreve o custo computacional de um algoritmo recursivo de ordenação.

9.1 Programação dinâmica: mochila 0/1

Considere n candidatos, numerados de 0 até $n-1$, cada um associado a um custo w_i e a um valor quantizado $v_i^{(q)}$.

Seja W o orçamento máximo de unidades lexicais.

Na implementação da Parte B, define-se a chave de otimização:

$$a_i = v_i^{(q)}(W+1) - w_i. \tag{40}$$

Essa chave prioriza o maior valor quantizado e, em caso de empate, o menor custo total da seleção.

Define-se $E(i, w)$ como a melhor chave acumulada possível considerando os candidatos a partir da posição i e a capacidade disponível w.

A condição de parada é:

$$E(n, w) = 0, \qquad 0 \leq w \leq W. \tag{41}$$

Para os demais estados, a recorrência é:

$$E(i, w) = \begin{cases} E(i+1, w), & w_i > w, \\ \max\left\{E(i+1, w), a_i + E(i+1, w - w_i)\right\}, & w_i \leq w. \end{cases} \tag{42}$$

Quando o candidato não cabe na capacidade disponível, a única possibilidade é excluí-lo.

Caso contrário, comparam-se as alternativas de exclusão e inclusão, preservando-se aquela que produz a maior chave acumulada.

A implementação utiliza uma tabela bidimensional preenchida iterativamente em ordem decrescente de i, caracterizando uma abordagem *bottom-up*.

Os valores de w são percorridos de 0 até W em cada linha.

Dessa forma, cada estado necessário é calculado uma única vez, evitando a repetição de subproblemas.

9.2 Relação com o custo computacional

A implementação inicializa uma matriz com $(n+1)(W+1)$ posições.

O preenchimento executa n iterações externas e $W+1$ iterações internas.

Para representar o custo de preenchimento das primeiras i linhas de candidatos, pode-se definir a recorrência temporal:

$$C(i, W) = C(i-1, W) + \Theta(W+1), \qquad i \geq 1. \tag{43}$$

Com condição inicial:

$$C(0, W) = \Theta(W+1), \tag{44}$$

considerando a inicialização da linha de base da matriz.

Expandindo a recorrência, obtém-se:

$$\begin{aligned} C(n, W) &= C(n-1, W) + \Theta(W+1) \\ &= C(n-2, W) + 2\Theta(W+1) \\ &= \cdots \\ &= \Theta((n+1)(W+1)). \end{aligned} \tag{45}$$

Portanto, o custo de inicialização e preenchimento da tabela é:

$$C(n, W) = \Theta((n+1)(W+1)). \tag{46}$$

Essa expressão descreve a etapa central da programação dinâmica.

O custo total da função também inclui a validação, a ordenação inicial e a reconstrução da solução, conforme a análise apresentada na Seção 7.

9.3 Recorrência temporal do Merge Sort

A recuperação lexical da configuração C3 utiliza Merge Sort para ordenar os P resultados com pontuação positiva.

O algoritmo divide o conjunto em duas partes aproximadamente iguais, ordena cada uma recursivamente e intercala as listas resultantes.

Para $P \geq 2$, sua recorrência temporal pode ser expressa por:

$$T(P) = T(\lfloor P/2 \rfloor) + T(\lceil P/2 \rceil) + \Theta(P). \tag{47}$$

Com condição inicial:

$$T(P) = \Theta(1), \qquad P \leq 1. \tag{48}$$

Quando P é uma potência de dois, a recorrência pode ser simplificada para:

$$T(P) = 2T(P/2) + \Theta(P). \tag{49}$$

Aplicando o Teorema Mestre, tem-se:

$$a = 2, \qquad b = 2, \qquad f(P) = \Theta(P). \tag{50}$$

Como:

$$P^{\log_b a} = P^{\log_2 2} = P, \tag{51}$$

aplica-se o segundo caso do Teorema Mestre, obtendo-se:

$$\boxed{T(P) = \Theta(P \log P).} \tag{52}$$

Esse resultado é coerente com a implementação recursiva examinada em `src/paa_context/merge_sort.py`.

A contagem prática de comparações, apresentada na Seção 7.5, constitui uma verificação complementar do comportamento esperado, sem substituir a demonstração matemática.

10. Metodologia experimental

10.1 Configurações comparadas

A avaliação experimental comparará duas estratégias de seleção de contexto, utilizando o mesmo conjunto de candidatos recuperados pela configuração C3 da AV1 (busca linear e Merge Sort).

Tabela 5: Estratégias previstas para comparação.

Estratégia	Descrição
Programação dinâmica	Seleção ótima de chunks por mochila 0/1, considerando relevância e orçamento.
Estratégia gulosa	Seleção sequencial de chunks pela razão relevância/custo, respeitando o orçamento máximo de unidades lexicais.

10.2 Cargas e repetições

O planejamento inicial considera 30 consultas herdadas da AV1, com orçamentos de 256, 512 e 1.024 unidades lexicais.

Serão considerados conjuntos de 10, 30 e 50 candidatos, respeitando a quantidade disponível de resultados com relevância positiva.

Cada cenário será executado pelo menos três vezes, quando o ambiente permitir.

Os valores poderão ser ajustados após a verificação da viabilidade experimental.

10.3 Métricas previstas

- Relevância total dos chunks selecionados.
- Quantidade de unidades lexicais utilizadas e respeito ao orçamento.
- Taxa de utilização do orçamento.
- Redundância entre os chunks selecionados.
- Tempo de seleção e tempo total do fluxo.
- Consumo aproximado de memória.

10.4 Reprodutibilidade

Os experimentos deverão registrar os parâmetros utilizados, a versão do corpus, a configuração de recuperação, as dependências, o ambiente computacional e o commit correspondente à AV2.

A recuperação será mantida fixa, garantindo que ambos os algoritmos recebam os mesmos candidatos e valores.

11. Resultados e gráficos

12. Discussão de escalabilidade

13. Relação com IA generativa e RAG

A seleção de contexto proposta na AV2 pode ser integrada à etapa de preparação de informações em sistemas de geração aumentada por recuperação (*Retrieval-Augmented Generation* — RAG).

Nesse cenário, os chunks recuperados da AV1 constituem candidatos à composição do contexto fornecido ao modelo de linguagem.

A AV2 acrescenta uma etapa de seleção sob orçamento, buscando maximizar a relevância lexical acumulada sem ultrapassar a capacidade estabelecida para os fragmentos.

A seleção ótima no modelo da mochila 0/1 não garante, isoladamente, maior qualidade das respostas, pois redundância, diversidade e relevância semântica não são diretamente representadas pela função objetivo.

14. Limitações e ameaças à validade

As principais limitações previstas para a abordagem são:

- **Relevância lexical:** o score TF-IDF depende da correspondência entre os termos da consulta e dos chunks, não representando diretamente similaridade semântica.
- **Aditividade:** a função objetivo soma relevâncias individuais, sem penalizar automaticamente informações redundantes.
- **Tokenização:** o custo dos chunks é obtido do campo `n_tokens`, calculado na fragmentação da AV1 a partir das palavras do texto normalizado, identificadas pela expressão regular `\w+`.

  A pontuação de relevância utiliza outro procedimento de normalização, implementado em `preprocessing.norm` que remove acentos.

  Portanto, o custo e a relevância não são necessariamente calculados sobre a mesma representação lexical. Além disso, as unidades lexicais utilizadas como custo não equivalem necessariamente aos tokens de um modelo de linguagem (LLM).
- **Corpus:** o estudo utiliza um único livro introdutório de programação, limitando a generalização dos resultados.
- **Recuperação:** a etapa de seleção depende dos candidatos produzidos pelo mecanismo lexical da AV1. Chunks relevantes não recuperados não podem ser selecionados posteriormente.
- **Validade experimental:** as conclusões dependerão dos orçamentos, da quantidade de candidatos, das consultas e do ambiente computacional utilizados.

A influência dessas limitações será discutida à luz dos resultados experimentais obtidos nas etapas posteriores.

15. Declaração de uso de IA generativa

A documentação de uso de IA generativa deverá ser mantida em `docs/USO_DE_IA.md`, com a ferramenta e o modelo efetivamente utilizados, finalidade, exemplos de prompts, sugestões aceitas ou rejeitadas e procedimentos de verificação. Não se atribui a um modelo específico uma contribuição sem que seu nome tenha sido registrado.

Nesta etapa, a IA generativa foi empregada como apoio à revisão textual e à estruturação metodológica da especificação. A verificação das formulações, dos critérios de corretude, dos casos de borda e dos contratos cabe à equipe, com base na implementação e nos testes que forem efetivamente realizados. A conclusão dos experimentos não é pressuposta neste documento.

> **Compromisso de uso crítico.** Propostas produzidas com auxílio de IA não constituem evidência de corretude por si só. A demonstração da programação dinâmica, os testes unitários e as medições experimentais deverão ser documentados nas respectivas etapas, com suas evidências verificáveis.

16. Contribuição individual

A divisão abaixo indica as responsabilidades acordadas para a AV2. As evidências concretas de execução, como commits, arquivos e resultados, deverão ser registradas em `docs/CONTRIBUICOES.md`, distinguindo atividades planejadas das concluídas.

Integrante	Responsabilidade na AV2
João Cosme Sena Sá	Parte B: implementação e corretude; Parte D: experimentos.
Eduardo Henrique do Lago Silva	Parte B: implementação e corretude; Parte C: complexidade; Parte D: experimentos.
Helena Carvalho Leal	Parte D: experimentos; Parte E: relação com IA generativa.
Hernandison da Silva Bispo	Parte A: especificação do problema; Parte E: relação com IA generativa.
Nadianne Maria dos Santos Galvão	Parte A: especificação do problema; Parte C: complexidade.
Rafael Takeguma Goto	Parte B: implementação e corretude; Parte E: relação com IA generativa.

17. Vídeo da atividade

AV2: gravação e URL ainda pendentes de confirmação. O vídeo e a apresentação deverão refletir as contribuições efetivamente realizadas, a implementação, as verificações de corretude e os resultados obtidos. A URL deverá ser registrada neste relatório, no `README.md` e no `VIDEO.md` quando estiver disponível.

18. Referências

Referências

[1] CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 4. ed. MIT Press, 2022.

[2] KLEINBERG, J.; TARDOS, É. *Algorithm Design*. Pearson/Addison-Wesley, 2006.

[3] SKIENA, S. S. *The Algorithm Design Manual*. 3. ed. Springer, 2020.

[4] SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Addison-Wesley, 2011.

[5] MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[6] LEWIS, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 2020. Disponível em: `https://arxiv.org/abs/2005.11401`.

[7] DOWNEY, A. B. *Think Python*. 3. ed. O'Reilly Media, 2024. Tradução brasileira: *Pense Python*, por Rodrigo Castelan Carlson. Disponível em: `https://rodrigocarlson.github.io/PensePython3ed/`.

[8] PEDREGOSA, F. et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011.